

CMPE 260 – Homework 3 Solutions

Your Name

March 28, 2025

Question 1 – Propositions and Predicate Calculus

Statements

1. John likes chocolate.
2. Mary likes chocolate.
3. John likes cake.
4. If someone likes both chocolate and cake, they are happy.

(1) Prolog Facts & Rules

```
% Facts
likes(john, chocolate).
likes(mary, chocolate).
likes(john, cake).

% Rule
happy(X) :-
    likes(X, chocolate),
    likes(X, cake).
```

(2) Query to Determine if John is Happy

```
?- happy(john).
```

Prolog checks `likes(john, chocolate)` and `likes(john, cake)`; both succeed.

(3) Query to Find All People Who Are Happy

```
?- happy(Person).
```

Only `Person = john` succeeds, given the closed-world assumption that Mary does not like cake unless stated.

(4) How Prolog Resolves the Queries

Resolution matches `happy(john)` with `happy(X)` and checks subgoals `likes(john, chocolate)` and `likes(john, cake)`.

(5) Effect of Removing the Closed-World Assumption

With the standard Prolog closed-world assumption, if Mary liking cake is not stated, it is taken as false. Removing the closed-world assumption might label it “unknown” in an extended framework, but standard Prolog still treats “unknown” as false.

Question 2 – Resolution and Proof by Contradiction

Given Facts

1. All dogs are mammals.
2. All mammals are animals.
3. Fido is a dog.
4. Cats are not dogs.
5. If an animal is a mammal and is not a cat, it is a dog.

(1) Prolog Knowledge Base

```
% 1. All dogs are mammals  
mammal(X) :- dog(X).
```

```
% 2. All mammals are animals  
animal(X) :- mammal(X).
```

```
% 3. Fido is a dog  
dog(fido).
```

```
% 4. Cats are not dogs (represented carefully)  
% For example:  
cat(X) :- \+ dog(X).
```

```
% 5. If an animal is a mammal and not a cat, it is a dog  
dog(X) :-  
    animal(X),  
    mammal(X),  
    \+ cat(X).
```

(2) Query to Prove Fido is an Animal

`?- animal(fido).`

Prolog finds `mammal(fido)` needed, then `dog(fido)`, which is stated. Hence `animal(fido)` is true.

(3) Proof by Contradiction for “If Fido is a Mammal and Not a Cat, Then Fido is a Dog”

Assume Fido is a mammal and not a cat, and also assume Fido is not a dog. Then the only way Fido can be a mammal is by being a dog (from the rules), which contradicts “Fido is not a dog.” Hence the assumption fails, proving Fido must be a dog.

(4) Resolution Steps

1. `mammal(fido) -> dog(fido).`
2. `\+ cat(fido)` is consistent with “not a cat.”
3. Contradiction arises if we also assume `\+ dog(fido)` (i.e. Fido is not a dog).

(5) Why Backtracking is Needed

Prolog may explore different predicates (`dog/1`, `cat/1`) or unify different variables. When a contradiction is reached, it backtracks to try alternative paths or confirm no solution exists.

Question 3 – Unification and Instantiation

Given Prolog KB

```
parent(bob, alice).
parent(alice, charlie).
father(X, Y) :- parent(X, Y), male(X).
male(bob).
```

(1) Query to Check if bob is Father of charlie

`?- father(bob, charlie).`

We need `parent(bob, charlie)` and `male(bob)` for it to succeed. If `parent(bob, charlie)` is not in the KB, it fails unless we add that fact.

(2) How Unification Works

Prolog attempts to unify `father(bob, charlie)` with `father(X, Y) :- parent(X, Y), male(X)` by setting `X = bob` and `Y = charlie`.

(3) Modify KB so `father/2` Works if `male(X)` is Missing

One possibility:

```
father(X, Y) :- parent(X, Y).
```

Or require `male(X)` only if it is known, but do not fail if it is unknown.

(4) Instantiation During Resolution

When calling `father(bob, charlie)`, `X` is instantiated to `bob`, and `Y` is instantiated to `charlie`. Prolog then checks the subgoals with these bindings.

Question 4 – Horn Clauses and Goal Resolution

Given Facts and Rules

```
likes(john, chocolate).
likes(mary, chocolate).
likes(john, cake).
happy(X) :- likes(X, chocolate), likes(X, cake).
```

(1) KB in Horn Clause Form

- **Facts (Headless Horn Clauses):**

```
likes(john, chocolate).
likes(mary, chocolate).
likes(john, cake).
```

- **Rule (Headed Horn Clause):**

```
happy(X) :- likes(X, chocolate), likes(X, cake).
```

(2) Headed vs Headless Clauses

- **Facts:** no body, hence headless.
- **Rule:** `happy(X) :- ...` has a head and a body.

(3) Prove john is Happy

`?- happy(john).`

Matches `happy(X)` with `john`, checks `likes(john, chocolate)` and `likes(john, cake)`.

(4) Modify happy/1 to Use OR

```
happy(X) :-  
    likes(X, chocolate);  
    likes(X, cake).
```

Now someone is happy if they like chocolate OR cake.

(5) Prolog Query Processing with Backtracking

For `happy(X)`, Prolog first tries `likes(X, chocolate)`. If it fails, it backtracks and tries `likes(X, cake)`.

Question 5 – Backtracking and Resolution

Given KB

```
sibling(X, Y) :- parent(Z, X), parent(Z, Y), X \= Y.  
parent(john, alice).  
parent(john, bob).  
parent(jane, alice).  
parent(jane, bob).
```

(1) Query to Check if alice and bob are Siblings

`?- sibling(alice, bob).`

(2) How Prolog Uses Backtracking

- Prolog matches `sibling(alice, bob)` with the rule.
- Tries `parent(Z, alice) = parent(john, alice)`. Then checks `parent(john, bob)`. Succeeds. Confirms `alice ≠ bob`. `Z = john` is a solution.
- On backtracking, it tries `Z = jane`, also a parent of both, yielding a second solution.

(3) Modify KB for Half-Siblings

```
half_sibling(X, Y) :-  
    parent(Z, X),  
    parent(Z, Y),  
    X \= Y,  
    \+ ( parent(W, X),  
         parent(W, Y),  
         W \= Z ).
```

(4) How Backtracking Changes

The extra check for `\+ ( ... )` causes Prolog to verify no other parent is shared. It may explore multiple parent possibilities and backtrack as needed.

Question 6 – List Structures and Pattern Matching

Define length/2 in Prolog

```
% Base case  
list_length([], 0).  
  
% Recursive case  
list_length([_Head | Tail], N) :-  
    list_length(Tail, M),  
    N is M + 1.
```

Testing

```
?- list_length([a, b, c, d], L).  
L = 4.  
  
?- list_length([1, 2, 3, 4, 5], L).  
L = 5.
```

Define member/2

```
list_member(X, [_Head | Tail]) :-  
    list_member(X, Tail).
```

How Matching and Resolution Work

Prolog unifies the list's head and tail in each recursive step. If the item is not the head, it tries the tail.

Fail After the First Found Solution

```
list_member_once(X, [X | _Tail]) :- !.  
list_member_once(X, [_Head | Tail]) :-  
    list_member_once(X, Tail).
```

Question 7 – Arithmetic in Prolog

Given KB

```
speed(ford, 100).  
speed(chevy, 105).  
speed(dodge, 95).  
speed(volvo, 80).  
  
time(ford, 20).  
time(chevy, 21).  
time(dodge, 24).  
time(volvo, 24).  
  
distance(X, Y) :-  
    speed(X, Speed),  
    time(X, Time),  
    Y is Speed * Time.
```

(1) Query for Chevy's Distance

```
?- distance(chevy, D).  
D = 2205.
```

(2) How is Operates

`Y is Speed * Time` evaluates the arithmetic expression and unifies the result with `Y`.

(3) Fail If Speed > 100

```
distance(X, Y) :-  
    speed(X, Speed),  
    Speed =< 100,  
    time(X, Time),  
    Y is Speed * Time.
```

(4) Compute Average Speed Across All Cars

A possible approach:

```
all_speeds(List) :-  
    findall(S, speed(_Car, S), List).
```

```
sum_list([], 0).  
sum_list([H | T], Sum) :-  
    sum_list(T, Rest),  
    Sum is H + Rest.
```

```
average_speed(Avg) :-  
    all_speeds(Speeds),  
    sum_list(Speeds, Sum),  
    length(Speeds, Count),  
    Count > 0,  
    Avg is Sum / Count.
```

Question 8 – Recursive List Operations

(1) append/3, reverse/2, delete/3

append/3:

```
my_append([], L, L).  
my_append([H | T], L2, [H | R]) :-  
    my_append(T, L2, R).
```

reverse/2 (classic version):

```
my_reverse([], []).  
my_reverse([H | T], R) :-  
    my_reverse(T, RT),  
    my_append(RT, [H], R).
```

delete/3:


```

my_delete(_, [], []).
my_delete(X, [X | T], T2) :-
    my_delete(X, T, T2).
my_delete(X, [H | T], [H | T2]) :-
    X \= H,
    my_delete(X, T, T2).

```

(2) Tests on Example Lists

```

?- my_append([1,2,3,4], [5,6], Result).
Result = [1,2,3,4,5,6].

```

```

?- my_reverse([a,b,c,d], R).
R = [d,c,b,a].

```

```

?- my_delete(3, [1,2,3,4], L).
L = [1,2,4].

```

(3) How Prolog Processes These with Backtracking

- `append/3` matches empty vs. `[H|T]` and recurses.
- `reverse/2` uses `append/3` each iteration to place the head at the end.
- `delete/3` tries two clauses: one where the head is the element, one where it's not.

Question 9 – Closed-World Assumption and Negation

(1) Define `knows/2` and Some Facts

```

knows(john, prolog).
knows(mary, prolog).
knows(mary, python).

```

(2) Query if Someone Knows Prolog

```

?- knows(Person, prolog).
Person = john ;
Person = mary ;
false.

```

(3) Rule with Negation

```
does_not_know(X, Y) :-  
    \+ knows(X, Y).
```

(4) Effect of the Closed-World Assumption

If `knows(alice, prolog)` is not in the KB, Prolog assumes it is false (negation as failure).
Hence `doesnotknow(alice, prolog)` succeeds.

Wrap-Up

These Prolog examples illustrate resolutions, backtracking, unification, and negation under the closed-world assumption. The code snippets should guide you in constructing your own solutions and explaining Prolog's behavior.